



Rapport de projet

SAE FindMyWord

Ressource : R2.01 - Base de la programmation objet
Université Sorbonne Paris Nord - IUT de Villetaneuse

Groupe	Cérès
Équipe	2 étudiants
Membres	AISSA DJELLOULI Badre ROY-MORAND Lilian
Projet	Jeu de devinette de mots en Java

1. Présentation du projet

Le projet FindMyWord consiste à développer en Java une version simplifiée d'un jeu inspiré de Wordle. Le joueur doit trouver un mot secret de cinq lettres en six tentatives maximum.

Après chaque tentative valide, le programme indique pour chaque lettre si elle est correcte et bien placée (OK), présente dans le mot secret mais mal placée (PRESENT), ou absente du mot secret (ABSENT).

Le programme vérifie aussi que chaque mot saisi respecte les règles imposées : exactement cinq lettres, uniquement des caractères alphabétiques et aucune lettre répétée. Une tentative invalide n'est pas comptabilisée comme un essai.

Notre version ajoute plusieurs fonctionnalités optionnelles : choix du nombre de manches, cumul des points sur les manches, chronométrage de chaque manche et historique final des parties.

2. Organisation du projet

Le projet est organisé dans le package Java findMyWord. Cette organisation permet de séparer clairement les classes du projet et de respecter la consigne de création de packages explicites.

```
SAEJava/  
├── src/  
│   ├── findMyWord/  
│   │   ├── FixedWordRepository.java  
│   │   ├── Game.java  
│   │   ├── Main.java  
│   │   ├── RandomWordRepository.java  
│   │   ├── Timer.java  
│   │   ├── Word.java  
│   │   └── WordRepository.java  
│   ├── data/  
│   │   └── mots.json  
│   ├── lib/  
│   │   └── wordset.jar  
│   └── bin/
```

Le dossier src/findMyWord contient le code source Java. Le dossier data contient le fichier mots.json, utilisé comme liste de mots. Le dossier lib contient la bibliothèque externe wordset.jar.

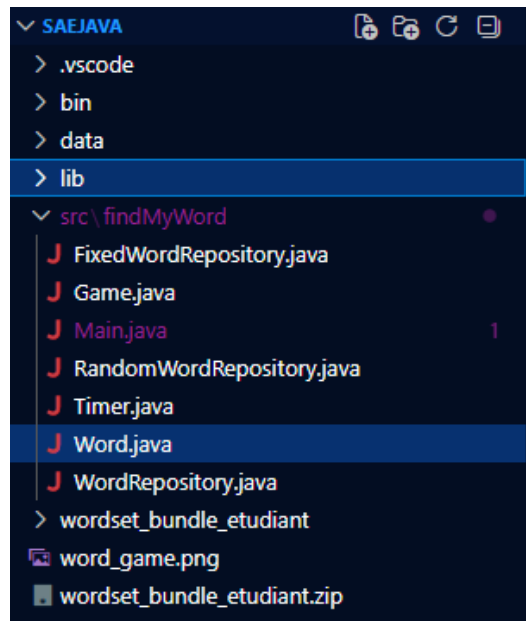


Figure 1 - Structure du projet dans VS Code.

3. Indications pour lancer le programme

Le programme doit être lancé depuis un terminal. Il n'est pas nécessaire d'ouvrir un IDE pour exécuter l'application.

Depuis le dossier racine du projet, la compilation sous Windows PowerShell se fait avec la commande suivante :

```
javac -cp ".;lib\wordset.jar" -d bin src\findMyWord\*.java
```

Cette commande compile toutes les classes Java du package findMyWord, utilise la bibliothèque wordset.jar et place les fichiers compilés dans le dossier bin.

```
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> javac -cp ".;lib\wordset.jar" -d bin src\findMyWord\*.java
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava>
```

Figure 2 - Compilation du projet depuis PowerShell.

Une fois la compilation terminée, le programme se lance sous Windows avec la commande suivante :

```
java -cp "bin;lib\wordset.jar" findMyWord.Main
```

Sous Linux et macOS, le séparateur du classpath est différent : il faut utiliser le caractère deux-points : au lieu du point-virgule ;.

La compilation sous Linux ou macOS peut donc se faire avec la commande suivante :

```
javac -cp ".:lib/wordset.jar" -d bin src/findMyWord/*.java
```

Puis le lancement sous Linux ou macOS peut se faire avec la commande suivante :

```
java -cp "bin:lib/wordset.jar" findMyWord.Main
```

```
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> javac -cp ".;lib\wordset.jar" -d bin src\findMyWord\*.java
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> java -cp "bin;lib\wordset.jar" findMyWord.Main

=====
FIND MY WORD - BUT1
=====

Saisissez votre nom : 
```

Figure 3 - Lancement du programme depuis le terminal.

Au démarrage, le programme affiche le titre du jeu, demande le nom du joueur, puis demande le nombre de manches souhaitées.

```
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> javac -cp ".;lib\wordset.jar" -d bin src\findMyWord\*.java
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> java -cp "bin;lib\wordset.jar" findMyWord.Main

=====
FIND MY WORD - BUT1
=====

Saisissez votre nom : badr

Bonjour badr !

Combien de manches voulez-vous jouer? 1, 5, 10 ou 20 : 
```

Figure 4 - Saisie du nom du joueur et choix du nombre de manches.

4. Modélisation UML

Le diagramme UML ci-dessous représente les classes principales du projet, leurs attributs, leurs méthodes et leurs relations. Nous avons choisi de n'intégrer qu'un seul diagramme UML dans le rapport afin de conserver une présentation claire et lisible.

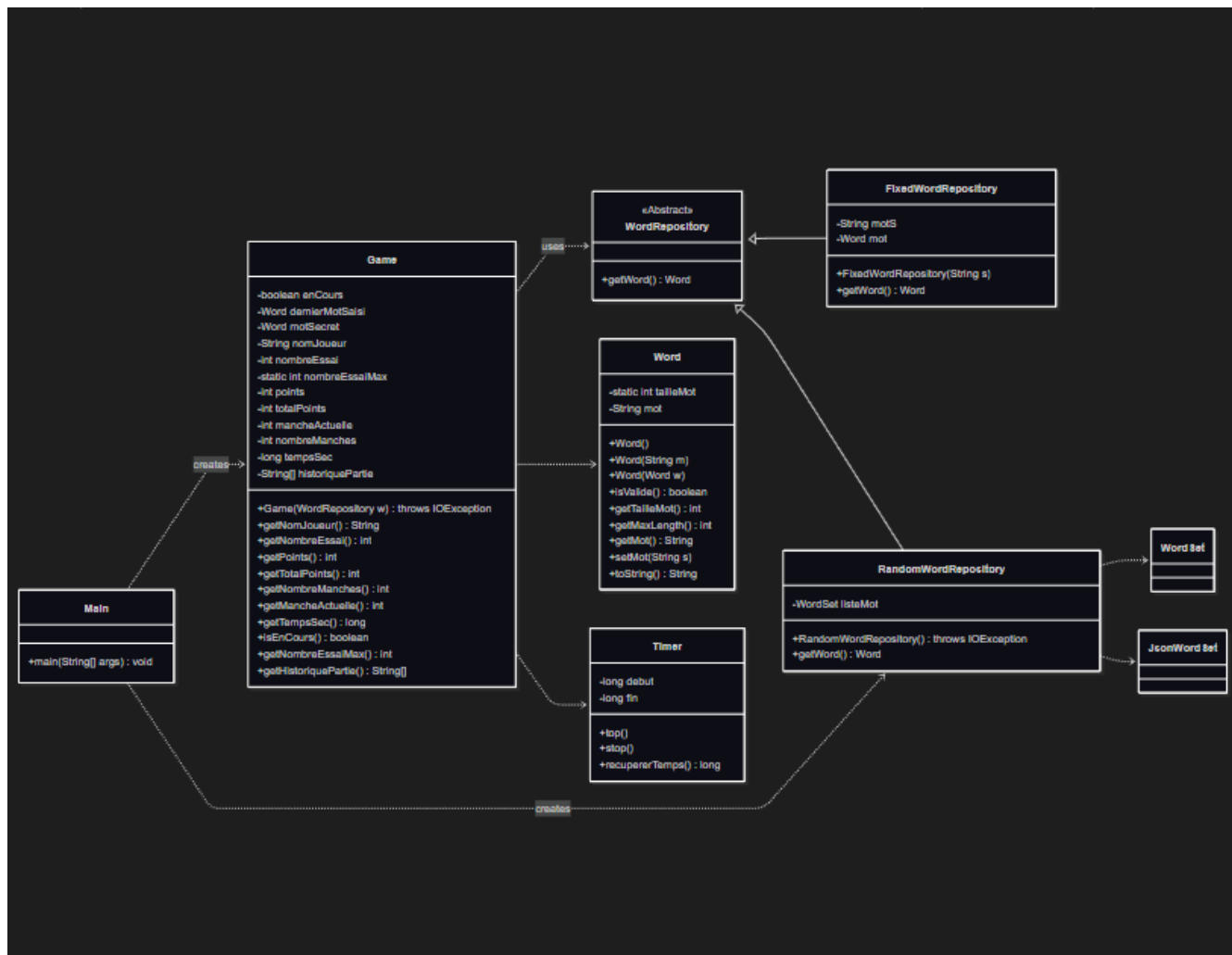


Figure 5 - Diagramme UML détaillé du projet FindMyWord.

4.1 Explication des principales classes

La classe Main sert à lancer le programme. Elle crée une source de mots puis démarre une partie en instanciant Game.

La classe Game gère la partie : affichage, saisies utilisateur, essais, analyse des propositions, score, temps, manches et historique final.

La classe Word représente un mot du jeu. Elle encapsule les règles de validité : cinq lettres, lettres alphabétiques uniquement et absence de lettre répétée.

La classe abstraite WordRepository définit une méthode abstraite getWord(). Les classes RandomWordRepository et FixedWordRepository en héritent et fournissent chacune une manière différente de récupérer un mot.

La classe Timer permet de mesurer le temps d'une manche.

5. Polymorphisme et choix de conception

L'un des choix principaux du projet est l'utilisation de la classe abstraite `WordRepository`. La classe `Game` ne dépend pas directement d'une implémentation précise. Elle reçoit simplement un objet de type `WordRepository`.

```
WordRepository repo = new RandomWordRepository();
Game jeu = new Game(repo);
```

Pour les tests, il est aussi possible d'utiliser un mot fixe :

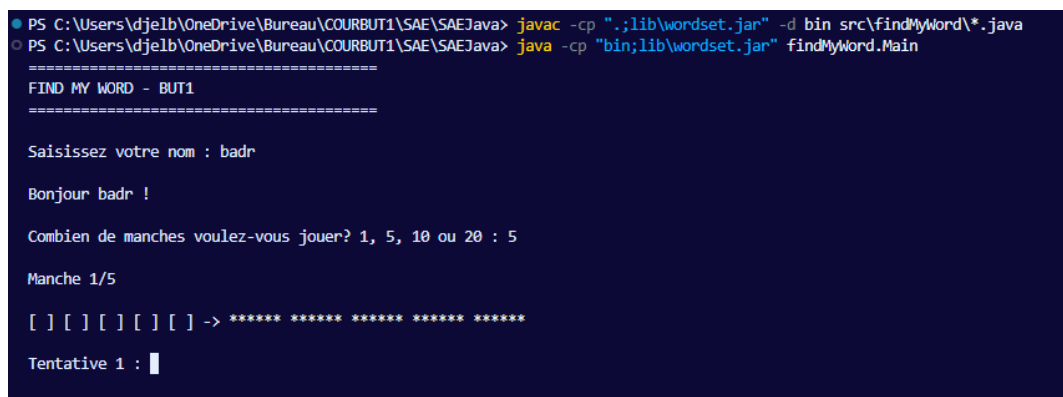
```
WordRepository repo = new FixedWordRepository("tales");
Game jeu = new Game(repo);
```

Dans les captures du rapport, le mot secret est fixé à `tales`. Cela permet de vérifier plus facilement le fonctionnement du jeu, les messages de victoire, les points et l'historique. Pour le fonctionnement normal, le projet possède aussi `RandomWordRepository`, qui récupère un mot depuis le fichier JSON.

6. Fonctionnalités réalisées

6.1 Choix du nombre de manches

Le joueur choisit le nombre de manches parmi 1, 5, 10 ou 20. Si l'entrée n'est pas correcte, le programme affiche une erreur et redemande une valeur.



```
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> javac -cp ".;lib\wordset.jar" -d bin src\findMyWord\*.java
PS C:\Users\djelb\OneDrive\Bureau\COURBUT1\SAE\SAEJava> java -cp "bin;lib\wordset.jar" findMyWord.Main

=====
FIND MY WORD - BUT1
=====

Saisissez votre nom : badr

Bonjour badr !

Combien de manches voulez-vous jouer? 1, 5, 10 ou 20 : 5

Manche 1/5

[ ] [ ] [ ] [ ] [ ] -> *****

Tentative 1 : 
```

Figure 6 - Choix de 5 manches et démarrage de la manche 1.

6.2 Validation des mots saisis

Le programme refuse les mots invalides et explique la raison du refus. Les tentatives invalides ne sont pas comptées : on reste sur la même tentative.



```
Manche 1/5

[ ] [ ] [ ] [ ] [ ] -> *****

Tentative 1 : chat

Erreur : le mot doit contenir exactement 5 lettres.

Tentative 1 : 
```

Figure 7 - Mot trop court : la tentative reste à 1.

```

Manche 1/5

[ ] [ ] [ ] [ ] [ ] -> ***** ***** ***** ***** *****

Tentative 1 : chat

Erreur : le mot doit contenir exactement 5 lettres.

Tentative 1 : ab1de

Erreur : le mot doit contenir uniquement des lettres.

Tentative 1 : █

```

Figure 8 - Caractère non alphabétique : message d'erreur précis.

```

Tentative 1 : belle

Erreur : le mot ne doit pas contenir de lettre répétée.

Tentative 1 : █

```

Figure 9 - Lettre répétée : message d'erreur précis.

6.3 Analyse des propositions

Lorsqu'un mot valide est saisi, le programme affiche l'historique des essais de la manche et indique pour chaque lettre le résultat de l'analyse : OK, PRESENT ou ABSENT.

```

Tentative 1 : trels

Historique des essais :
[ T ][ R ][ E ][ L ][ S ] -> OK ABSENT PRESENT PRESENT OK

-----

Tentative 2 : █

```

Figure 10 - Exemple d'analyse d'une proposition valide.

6.4 Victoire, score et temps

Lorsque le joueur trouve le mot secret, le programme affiche un message de victoire, le nombre d'essais, les points obtenus pendant la manche, le total des points accumulés et le temps de la manche.

Les points s'accumulent sur les manches. Par exemple, une victoire au premier essai rapporte 6 points, et ces points s'ajoutent au total déjà obtenu pendant les manches précédentes.

```
Tentative 1 : tales

Historique des essais :
[ T ][ A ][ L ][ E ][ S ] -> OK OK OK OK OK

-----

Bravo badr !
Vous avez trouvé le mot en 1 essai(s).
Vous avez obtenu : 6 point(s) à cette manche.
Total des points : 17 point(s).
Temps pour cette manche : 1 secondes
```

Figure 11 - Victoire avec le mot fixe tales, score et temps affichés.

```
Tentative 1 : tales

Historique des essais :
[ T ][ A ][ L ][ E ][ S ] -> OK OK OK OK OK

-----

Bravo badr !
Vous avez trouvé le mot en 1 essai(s).
Vous avez obtenu : 6 point(s) à cette manche.
Total des points : 29 point(s).
Temps pour cette manche : 1 secondes

Voulez vous voir un historique des manches? oui - non : |
```

Figure 12 - Accumulation des points et demande d'affichage de l'historique.

6.5 Historique final des manches

À la fin de toutes les manches, le programme demande au joueur s'il souhaite consulter l'historique. Si le joueur répond oui, le programme affiche pour chaque manche le joueur, le mot, le numéro de manche, le résultat, le nombre d'essais, les points et le temps.

```
Voulez vous voir un historique des manches? oui - non : oui

Voici l'historique de vos manches :
```

Joueur : badr	Mot : tales	Manche : 1	Résultat : victoire	Essais : 2	Points : 5	Temps : 56 secondes
Joueur : badr	Mot : tales	Manche : 2	Résultat : victoire	Essais : 1	Points : 6	Temps : 0 secondes
Joueur : badr	Mot : tales	Manche : 3	Résultat : victoire	Essais : 1	Points : 6	Temps : 1 secondes
Joueur : badr	Mot : tales	Manche : 4	Résultat : victoire	Essais : 1	Points : 6	Temps : 17 secondes
Joueur : badr	Mot : tales	Manche : 5	Résultat : victoire	Essais : 1	Points : 6	Temps : 1 secondes

Figure 13 - Historique final des 5 manches.

7. Cas de test réalisés

Plusieurs tests ont été effectués afin de vérifier le bon fonctionnement du programme.

Test	Entrée / situation	Résultat observé
Lancement terminal	Compilation puis exécution avec java prog	Le programme compile et se lance correctement.
Mot trop court	chat	Erreur : le mot doit contenir exactement 5 lettres.
Caractère non alphabétique	é!@#	Erreur : le mot doit contenir uniquement des lettres.
Lettre répétée	belle	Erreur : le mot ne doit pas contenir de lettre répétée.

Tentative valide	trels	Analyse avec OK / PRESENT / ABSENT.
Victoire	tales	Victoire détectée, points et temps affichés.
Plusieurs manches	5 manches	Les points s'accumulent et l'historique final est affiché.

8. Bilan

8.1 Ce qui a été fait

- Projet Java structuré avec le package findMyWord.
- Classes principales créées : Main, Game, Word, WordRepository, RandomWordRepository, FixedWordRepository et Timer.
- Utilisation d'une classe abstraite WordRepository et du polymorphisme.
- Récupération possible d'un mot aléatoire depuis le JSON et utilisation d'un mot fixe pour les tests.
- Validation des mots avec messages d'erreur précis.
- Tentatives invalides non comptabilisées.
- Analyse des lettres avec OK / PRESENT / ABSENT.
- Gestion de la victoire et de la défaite.
- Système de points par manche et total cumulé.
- Choix du nombre de manches parmi 1, 5, 10 ou 20.
- Chronométrage des manches avec la classe Timer.
- Historique final des manches.
- Diagramme UML détaillé et lancement depuis terminal.

8.2 Ce qui n'a pas été fait

Le mode deux joueurs n'a pas été réalisé. Nous avons choisi de nous concentrer sur un mode un joueur complet, avec plusieurs manches, score cumulé, temps et historique.

L'interface graphique n'a pas été développée. Le programme fonctionne en mode texte dans le terminal.

8.3 Ce qui fonctionne

Le programme compile et se lance depuis le terminal. La saisie du nom, le choix des manches, la validation des mots, l'analyse des lettres, le calcul des points, le chronométrage et l'historique final fonctionnent correctement.

8.4 Limites connues

La principale limite du code est que la classe Game contient une grande partie de la logique directement dans son constructeur. Une amélioration possible serait de découper ce constructeur en méthodes plus courtes, par exemple jouerManche(), analyserMot(), afficherHistorique() ou demanderNombreManches().

L'historique est affiché en fin de partie, mais il n'est pas sauvegardé dans un fichier externe.

9. Organisation du travail

Le projet a été réalisé en binôme par AISSA DJELLOULI Badre et ROY-MORAND Lilian.

Badre AISSA DJELLOULI a principalement réalisé la classe Game, qui contient la base principale du fonctionnement du jeu : déroulement de la partie, essais, affichage, validation des propositions et calcul du score.

Lilian ROY-MORAND a travaillé sur le système de manches, la classe Timer permettant de mesurer le temps, ainsi que sur l'historique des manches.

La classe RandomWordRepository a été réalisée à deux : Badre a commencé la base, puis la classe a été vérifiée et améliorée avec Lilian.

Les autres classes ont été réfléchies et codées ensemble, lors d'appels Discord et de séances de travail en présentiel. Le diagramme UML a également été réalisé en commun.

Badre s'est occupé de mettre le projet sur GitHub afin de faciliter le travail à deux, le partage du code et les modifications successives.

10. Difficultés rencontrées et choix techniques

Une difficulté importante a été de bien comprendre l'architecture attendue, notamment le rôle de la classe abstraite WordRepository. Ce choix a permis de mettre en place le polymorphisme et de séparer la logique du jeu de la source des mots.

Une autre difficulté a été l'utilisation de wordset.jar et du fichier mots.json. Il fallait comprendre le rôle du classpath afin de compiler et lancer le programme correctement depuis le terminal.

La gestion des mots invalides a également demandé une vérification précise, car le programme devait afficher une erreur claire et ne pas compter l'essai.

Nous avons choisi d'utiliser FixedWordRepository pour les captures de test, avec le mot fixe tales. Cela permet de vérifier facilement la victoire, les points et l'historique. Pour le fonctionnement normal, RandomWordRepository permet de récupérer un mot aléatoire depuis le fichier JSON.

11. Conclusion

Ce projet nous a permis de mettre en pratique plusieurs notions importantes de programmation orientée objet : encapsulation, héritage, polymorphisme et séparation des responsabilités.

Le programme réalisé permet de jouer à une version simplifiée de FindMyWord depuis un terminal. Il gère les propositions du joueur, les erreurs de saisie, les essais, les résultats OK / PRESENT / ABSENT, les points, le temps, les manches et l'historique final.

Le mode deux joueurs n'a pas été implémenté, mais le programme intègre plusieurs fonctionnalités optionnelles appréciées : chronométrage, choix du nombre de manches, cumul des points et historique des parties.

Le projet est fonctionnel, respecte les principales contraintes du sujet et illustre le polymorphisme grâce à la classe abstraite WordRepository.